A list of challenges that you faced & how did you solve them.

1.Installing HDFS & Java in Docker Containers and Sharing Configuration Files with Windows

Challenge:
Installing HDFS and Java inside Docker containers and ensuring that Hadoop configuration files could be accessed and modified from Windows through a shared folder.

Solution:

In the **Dockerfile**, I:

a. Downloaded and installed all required dependencies.
b. Installed **Java 17**.
c. Downloaded and configured Hadoop.
d. Copied the start-cluster.sh script into the container.
e. Configured the container to automatically execute the script at startup so that each node starts its required Hadoop services automatically.

In docker-compose.yaml, I initially configured each node to build its image individually.

I used the following commands:

docker compose build

docker compose up

This ensured:

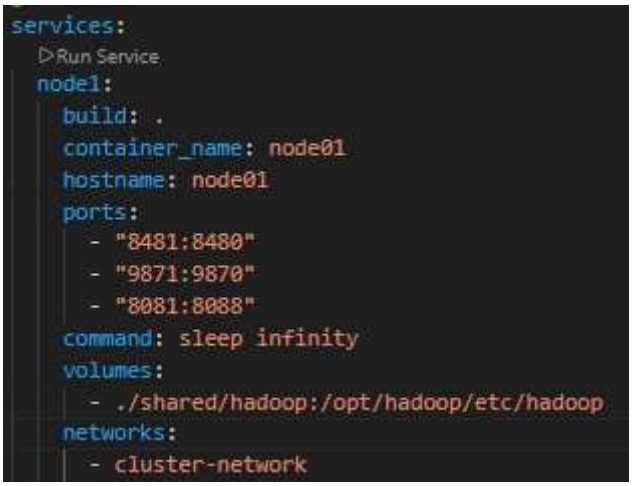
- Java and HDFS were installed automatically on all nodes.
- I verified installation using:
    - java –version
    - hdfs version

To allow configuration files to be shared between Windows and containers:

- I copied Hadoop configuration files from the container:

docker cp node01:/opt/hadoop/etc/hadoop ./shared/

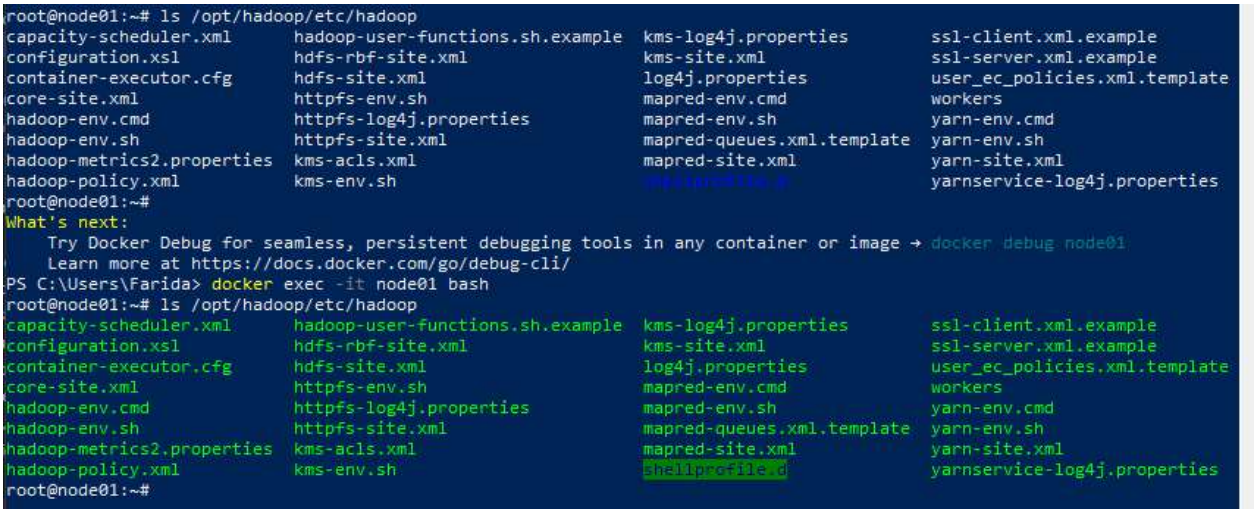
And add the path of shared file in volumes to be mounted.



Then restarted containers:

docker compose down
docker compose up -d

This enabled any changes made in the Windows shared folder to automatically reflect inside all nodes.



2.Excessive Disk Usage Due to Multiple Image Builds

Challenge:

Because each container was building its own image, every node created a separate image of approximately **5.5 GB**.

With 5 nodes, this resulted in:

5.5 GB × 5 = 27.5 GB

This storage consumption occurred even before uploading any data to HDFS, which would require additional storage.

Solution:

Since all nodes use the same dependencies and configuration, I optimized the setup:

- Only node01 builds a single image named: **hadoop-ha**
- All other nodes use this same image instead of building separate ones.

This optimization reduced total disk usage from:

27.5 GB → 5.5 GB

making the system significantly more efficient.

3.NameNode High Availability – JournalNode Connection Issue

Challenge:
While testing High Availability (HA) for the NameNodes, I encountered a connectivity issue:

> root@node02:~# hdfs haadmin -getServiceState nn2 2026-02-26 15:36:03,275 INFO ipc.Client: Retrying connect to server: node02/172.18.0.6:8020. Already tried 0 time(s); retry policy is RetryUpToMaximumCountWithFixedSleep(maxRetries=1, sleepTime=1000 MILLISECONDS) Operation failed: Call From node02/172.18.0.6 to node02:8020 failed on connection exception: java.net.ConnectException: Connection refused; For more details see: http://wiki.apache.org/hadoop/ConnectionRefused

The NameNode was unable to connect to the JournalNodes.

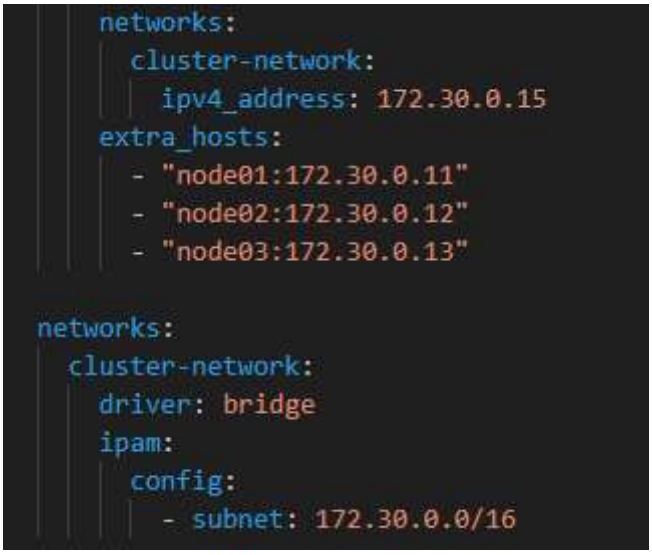
Root Cause:
The issue was related to Docker network resolution and dynamic container IP addresses.

Solution:

- Assigned **static IP addresses** to all nodes within the Docker network.
- Added extra_hosts entries in docker-compose.yaml to explicitly define the IP addresses of the nodes.

This ensured:

- Proper hostname resolution.
- Stable communication between NameNodes and JournalNodes.
- Successful High Availability configuration.



4.Stoping , Starting the containers & ordering the processes

Challenge:
When the containers start to run the start-cluster.sh some processes have to start before some processes and get connection refused errors:

- DataNodes cannot start before Active NN
- Standby NN cannot bootstrap before Active NN
- HA depends on coordination

Solution:

```
wait_for_namenode() {
    echo "Waiting for Active NameNode (nn1) to become reachable..."
    until hdfs haadmin -getServiceState nn1 >/dev/null 2>&1; do
        sleep 5
    done
    echo "Active NameNode is reachable."
}
```

This function waits until the Active NameNode (nn1) becomes reachable.

When I stop containers and starting again the script starts a processes which is already running:

Solution:

```
start_if_not_running() {
    local service=$1
    local cmd=$2
    if ! jps | grep -q "$service"; then
        echo "Starting $service..."
        $cmd
    else
        echo "$service is already running."
    fi
}
```

The function checks if the service is already running and starts it only if not running